

Chapter 9: Application Development

Outline

- Application Security
- Encryption and Its Applications

Application Security

SQL Injection

- Suppose query is constructed using
 - "select * from instructor where name = ' " + name + " ' "
- Suppose the user, instead of entering a name, enters:
 - X' or ' Y' = ' Y
- then the resulting statement becomes:
 - "select * from instructor where name = ' " + "X' or ' Y' = ' Y" + " ' "
 - which is:
 - select * from instructor where name = ' X' or ' Y' = ' Y'
 - User could have even used
 - X' ; update instructor set salary = salary + 10000; --
- Prepared statement internally uses:
"select * from instructor where name = ' X\ ' or \ ' Y\ ' = \ ' Y'
- **Always use prepared statements, with user inputs as parameters**
- Is the following prepared statemen secure?
 - conn.prepareStatement("select * from instructor where name = ' " + name + " ' ")

Cross Site Scripting

- HTML code on one page executes action on another page
 - E.g., `<img src = http://mybank.com/transfermoney?amount=1000&toaccount=14523>`
 - Risk: if user viewing page with above code is currently logged into mybank, the transfer may succeed
 - Above example simplistic, since GET method is normally not used for updates, but if the code were instead a script, it could execute POST methods
- Above vulnerability called **cross-site scripting (XSS)** or **cross-site request forgery (XSRF or CSRF)**
- **Prevent your web site from being used to launch XSS or XSRF attacks**
 - Disallow HTML tags in text input provided by users, using functions to detect and strip such tags
- **Protect your web site from XSS/XSRF attacks launched from other sites**
 - ..next slide

Cross Site Scripting

Protect your web site from XSS/XSRF attacks launched from other sites

- Use **referer** value (URL of page from where a link was clicked) provided by the HTTP protocol, to check that the link was followed from a valid page served from same site, not another site
- Ensure IP of request is same as IP from where the user was authenticated
 - Prevents hijacking of cookie by malicious user
- Never use a GET method to perform any updates
 - This is actually recommended by HTTP standard

Password Leakage

- Never store passwords, such as database passwords, in clear text in scripts that may be accessible to users
 - E.g., in files in a directory accessible to a web server
 - Normally, web server will execute, but not provide source of script files such as file.jsp or file.php, but source of editor backup files such as file.jsp~, or .file.jsp.swp may be served
- Restrict access to database server from IPs of machines running application servers
 - Most databases allow restriction of access by source IP address

Application Authentication

- Single factor authentication such as passwords too risky for critical applications
 - Guessing of passwords, sniffing of packets if passwords are not encrypted
 - Passwords reused by user across sites
 - Spyware which captures password
- Two-factor authentication
 - E.g., password plus one-time password sent by SMS
 - E.g., password plus one-time password devices
 - Device generates a new pseudo-random number every minute, and displays to user
 - User enters the current number as password
 - Application server generates same sequence of pseudo-random numbers to check that the number is correct.

Application Authentication

- **Man-in-the-middle** attack
 - E.g., web site that pretends to be mybank.com, and passes on requests from user to mybank.com, and passes results back to user
 - Even two-factor authentication cannot prevent such attacks
- Solution: authenticate Web site to user, using digital certificates, along with secure http protocol
- **Central authentication** within an organization
 - Application redirects to central authentication service for authentication
 - Avoids multiplicity of sites having access to user's password
 - LDAP or Active Directory used for authentication

Single Sign-On

- **Single sign-on** allows user to be authenticated once, and applications can communicate with authentication service to verify user's identity without repeatedly entering passwords
- **Security Assertion Markup Language (SAML)** standard for exchanging authentication and authorization information across security domains
 - E.g., user from Yale signs on to external application such as acm.org using userid [joe@yale.edu](#)
 - Application communicates with Web-based authentication service at Yale to authenticate user, and find what the user is authorized to do by Yale (e.g., access certain journals)
- **OpenID** standard allows sharing of authentication across organizations
 - E.g., application allows user to choose Yahoo! as OpenID authentication provider, and redirects user to Yahoo! for authentication

Application-Level Authorization

- Current SQL standard does not allow fine-grained authorization such as “students can see their own grades, but not other’s grades”
 - Problem 1: Database has no idea who are application users
 - Problem 2: SQL authorization is at the level of tables, or columns of tables, but not to specific rows of a table
- One workaround: use views such as

```
create view studentTakes as  
select *  
from takes  
where takes.ID = syscontext.user_id()
```

 - where *syscontext.user_id()* provides end user identity
 - End user identity must be provided to the database by the application
 - Having multiple such views is cumbersome

Application-Level Authorization (Cont.)

- Currently, authorization is done entirely in application
- Entire application code has access to entire database
 - Large surface area, making protection harder
- Alternative: **fine-grained (row-level) authorization** schemes
 - Extensions to SQL authorization proposed but not currently implemented
 - Oracle Virtual Private Database (VPD) allows predicates to be added transparently to all SQL queries, to enforce fine-grained authorization
 - E.g., add *ID= sys_context.user_id()* to all queries on student relation if user is a student

Audit Trails

- Applications must log actions to an audit trail, to detect who carried out an update, or accessed some sensitive data
- Audit trails used after-the-fact to
 - Detect security breaches
 - Repair damage caused by security breach
 - Trace who carried out the breach
- Audit trails needed at
 - Database level, and at
 - Application level

Encryption

- Data may be *encrypted* when database authorization provisions do not offer sufficient protection.
- Properties of good encryption technique:
 - Relatively simple for authorized users to encrypt and decrypt data.
 - Encryption scheme depends not on the secrecy of the algorithm but on the secrecy of a parameter of the algorithm called the encryption key.
 - Extremely difficult for an intruder to determine the encryption key.
- **Symmetric-key encryption**: same key used for encryption and for decryption
- **Public-key encryption** (a.k.a. **asymmetric-key encryption**): use different keys for encryption and decryption
 - Encryption key can be public, decryption key secret

Encryption (Cont.)

- **Data Encryption Standard (DES)** substitutes characters and rearranges their order on the basis of an encryption key which is provided to authorized users via a secure mechanism. Scheme is no more secure than the key transmission mechanism since the key has to be shared.
- **Advanced Encryption Standard (AES)** is a new standard replacing DES, and is based on the Rijndael algorithm, but is also dependent on shared secret keys.
- **Public-key encryption** is based on each user having two keys:
 - *Public key* – publicly published key used to encrypt data, but cannot be used to decrypt data
 - *Private key* -- key known only to individual user, and used to decrypt data. Need not be transmitted to the site doing encryption.

Encryption scheme is such that it is impossible or extremely hard to decrypt data given only the public key.

- The RSA public-key encryption scheme is based on the hardness of factoring a very large number (100's of digits) into its prime components

Encryption (Cont.)

- **Hybrid schemes** combining public key and private key encryption for efficient encryption of large amounts of data
- Encryption of small values such as identifiers or names vulnerable to **dictionary attacks**
 - Especially if encryption key is publicly available
 - But even otherwise, statistical information such as frequency of occurrence can be used to reveal content of encrypted data
 - Can be deterred by adding extra random bits to the end of the value, before encryption, and removing them after decryption
 - Same value will have different encrypted forms each time it is encrypted, preventing both above attacks
 - Extra bits are called **salt bits**

Encryption in Databases

- Database widely support encryption
- Different levels of encryption:
 - **disk block**
 - Every disk block encrypted using key available in database-system software.
 - Even if attacker gets access to database data, decryption cannot be done without access to the key.
 - **Entire relations, or specific attributes of relations**
 - Non-sensitive relations, or non-sensitive attributes of relations need not be encrypted
 - However, attributes involved in primary/foreign key constraints cannot be encrypted.
- Storage of encryption or decryption keys
 - Typically, single master key used to protect multiple encryption/decryption keys stored in database
- Alternative: encryption/decryption is done in application, before sending values to the database

Encryption and Authentication

- Password based authentication is widely used, but is susceptible to sniffing on a network.
- **Challenge-response** systems avoid transmission of passwords
 - DB sends a (randomly generated) challenge string to user.
 - User encrypts string and returns result.
 - DB verifies identity by decrypting result
 - Can use public-key encryption system by DB sending a message encrypted using user's public key, and user decrypting and sending the message back.
- **Digital signatures** are used to verify authenticity of data
 - E.g., use private key (in reverse) to encrypt data, and anyone can verify authenticity by using public key (in reverse) to decrypt data. Only holder of private key could have created the encrypted data.
 - Digital signatures also help ensure **nonrepudiation**: sender cannot later claim to have not created the data

End of Chapter 9

Digital Certificates

- **Digital certificates** are used to verify authenticity of public keys.
- Problem: when you communicate with a web site, how do you know if you are talking with the genuine web site or an imposter?
 - Solution: use the public key of the web site
 - Problem: how to verify if the public key itself is genuine?
- Solution:
 - Every client (e.g., browser) has public keys of a few root-level **certification authorities**
 - A site can get its name/URL and public key signed by a certification authority: signed document is called a **certificate**
 - Client can use public key of certification authority to verify certificate
 - Multiple levels of certification authorities can exist. Each certification authority
 - Presents its own public-key certificate signed by a higher level authority, and
 - Uses its private key to sign the certificate of other web sites/authorities